
Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-extension-guidelines-00
Published:	14 November 2026
Intended Status:	Informational
Expires:	18 May 2027
Authors:	M. Wullink P. Kowalik SIDN Labs DENIC

Extension Guidelines for RESTful Provisioning Protocol (RPP)

Abstract

The RESTful Provisioning Protocol (RPP) defines a base set of data objects, component objects, and operations for provisioning registry data. This document defines guidelines and a normative mechanism for extension specifications that add data elements, associations, operations, result codes, or other functionality to objects and operations defined in a base RPP specification, without modifying the base specification itself. It specifies the recognized RPP extension points, the JSON Schema composition method extensions MUST use to remain independently combinable, the corresponding registration requirements, and considerations for client and server implementations that support one or more extensions.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Terminology	4
3. Conventions Used in This Document	5
4. Restrictions on Extensions	5
5. RPP Extension Points	5
5.1. New Data Objects	5
5.2. New Data Elements on Existing Objects	5
5.3. New Operations	5
5.4. URL Endpoints	6
5.5. Operation Inputs and Outputs	6
5.6. External Data Types	6
5.7. New RPP Result Codes	6
5.8. Error Object Extension Fields	6
5.9. Additional Query Parameters	6
5.10. New Authentication and Authorization Methods	7
5.11. Discovery document	7
6. Extending Data Objects	7
6.1. JSON Schema Composition	7
6.2. Schema Identification	7
6.3. Consumer Implementation	7
6.3.1. Java	8
6.4. Registration	8
6.5. Validation	8
7. Extension Specification Format	9
8. IANA Considerations	9
9. Internationalization Considerations	9
10. Privacy Considerations	10
11. Change History	10

12. References	10
12.1. Normative References	10
12.2. Informative References	10
Appendix A. Example Extension	10
A.1. Extension Overview	11
A.2. Changed Data Objects	11
A.2.1. Domain Name Data Object	11
A.3. New Data Objects	11
A.4. JSON Schema	11
A.5. Examples	12
A.6. Composition and Validation	13
A.7. IANA Considerations	13
A.8. Security Considerations	13
A.9. Privacy Considerations	13
A.10. Internationalization Considerations	14
Acknowledgements	14
Authors' Addresses	14

1. Introduction

The RESTful Provisioning Protocol (RPP), as defined in [\[I-D.ietf-rpp-core\]](#), [\[I-D.ietf-rpp-data-objects\]](#), and [\[I-D.ietf-rpp-json\]](#), provides a base set of data objects, component objects, and operations for the provisioning of registry data. Deployments may need to add functionality beyond this base set, for example to support policies specific to a registry, a jurisdiction, or a class of registered objects. RPP is designed to accommodate such needs through extension specifications, so that additional functionality can be introduced without modifying the base specification that defines the object or operation being extended.

Because independently developed extension specifications may be deployed together, and because RPP relies on JSON Schema, as defined in [\[I-D.ietf-rpp-json\]](#), to validate protocol messages, extensions need to follow a common, predictable pattern. Without such a pattern, extensions that are individually valid could conflict when combined at a given deployment, or could require ad-hoc, per-combination schema and code to be written for every deployment profile. This document defines that common pattern.

This document identifies the set of extension points along which an RPP base specification MAY be extended, including new data objects, new data elements on existing objects, new operations, new operation inputs and outputs, external data types, new RPP result codes,

error object extension fields, additional query parameters, and additions to the RPP Discovery document. For each extension point, this document states the applicable registration requirements.

This document further discusses considerations for client and server implementations that need to compose, validate, and process RPP messages that may include data contributed by more than one extension.

This document does not itself define any RPP extension. It defines the rules that extension specifications, and the deployments that support them, are expected to follow.

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [\[RFC3986\]](#).

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

Base specification - The RPP specification, such as [\[I-D.ietf-rpp-data-objects\]](#), [\[I-D.ietf-rpp-json\]](#) or [\[I-D.ietf-rpp-core\]](#), that originally defines a data object, component object, or operation.

Extension specification - A specification that adds new data elements, associations, or operations to an object or operation defined in a base specification, without modifying the base specification itself.

Data object - A top-level provisioned resource object that has an independent lifecycle and identity, and defines the operations that can be performed on it, as defined in [\[I-D.ietf-rpp-data-objects\]](#).

Component object - A reusable data structure that carries data only, has no operations of its own, and is embedded within data objects or other objects, as defined in [\[I-D.ietf-rpp-data-objects\]](#).

Process object - An object that represents a long-running or multi-step operation initiated on a data object, and whose lifecycle is bound to that data object, as defined in [\[I-D.ietf-rpp-data-objects\]](#).

Data element - A logical unit of information of an object, identified by a stable name and defined by its cardinality, mutability, data type and constraints, as defined in [\[I-D.ietf-rpp-data-objects\]](#). In the JSON representation, a data element is represented as a property.

Transient data element - A data element that is part of the input or output of an operation but is not persisted as part of the state of an object.

Base object - A data object, component object or process object defined in a base specification, that is changed by one or more extension specifications.

Consumer - A server or client that generates, processes or validates RPP messages.

Deployment - A server together with the set of extensions it supports.

Profile - A named set of protocol features, versions and extensions that defines the capabilities of an RPP server, as defined in [\[I-D.ietf-rpp-core\]](#).

Discovery document - The JSON document, published by an RPP server at a well-known location, that describes the capabilities of the server, including the extensions it supports, as defined in [\[I-D.ietf-rpp-core\]](#).

JSON Schema - A vocabulary for describing and validating JSON documents, as defined in [\[JSON-SCHEMA\]](#). This document uses JSON Schema draft 2020-12.

`rpp:extends` - The property of an extension JSON Schema that maps the reference of each base object definition to the local definition in the extension schema that contributes the new properties, as defined in [\[I-D.ietf-rpp-json\]](#).

Effective schema - The JSON Schema that is composed, for one base object definition, from that definition and the definitions of all extensions that a deployment supports, using the `rpp:extends` mappings of the extension schemas. The effective schema has its own `$id`. The effective schema is also referred to as the combined schema.

Combined schema - The JSON Schema produced by composing the JSON Schema definition of a base object with the JSON Schema contributed by every extension specification supported by a given deployment or profile, as defined in [\[I-D.ietf-rpp-core\]](#). See effective schema.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [\[RFC2119\]](#).

In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

4. Restrictions on Extensions

An extension MUST NOT remove, redefine, or otherwise alter the semantics of a property defined by the RPP base specification. An extension MUST only add new properties or associations, and MUST ensure that any new elements do not conflict with existing ones in the base specification.

TODO: do we recommend to use namespaces, using prefixes, to avoid naming conflicts between properties added by different extensions?

5. RPP Extension Points

An extension specification MAY extend RPP along one or more of the following independent extension points. Each extension point has its own registration requirements and, where applicable, its own schema-composition method.

5.1. New Data Objects

Extensions MAY define entirely new resource, component, or process objects, as described in [\[I-D.ietf-rpp-data-objects\]](#).

5.2. New Data Elements on Existing Objects

Extensions MAY add new data elements (properties) to a data object, component object, or process object defined in a base specification. This is the extension point covered in detail in [Section 6](#) below.

5.3. New Operations

Extensions MAY define entirely new operations on existing or new data objects.

5.4. URL Endpoints

RPP endpoint URLs are not an independent extension point. As defined in [\[I-D.ietf-rpp-core\]](#), every endpoint URL and HTTP method is derived mechanically from a Data Object's "Identifier", from its operations, and from the Direct Access flag on its data elements; no endpoint is defined independently of a corresponding Data Object. Consequently:

- A new Data Object automatically obtains a collection endpoint once its "Identifier" is registered, by applying the `plural()` derivation rule.
- A new association with Direct Access set to `true` automatically obtains a sub-resource path, derived recursively from the parent resource's path.
- A new operation automatically obtains its HTTP method and URL path once its "Identifier" is registered.

An extension specification **MUST NOT** define ad-hoc endpoint URLs or HTTP methods. It **MUST** instead register the underlying data object, association, or operation as described in the corresponding extension point above, and rely on the derivation rules in [\[I-D.ietf-rpp-core\]](#) to determine the resulting endpoint.

5.5. Operation Inputs and Outputs

Extensions **MAY** add both transient and persistent data elements to the input or output of an existing operation.

5.6. External Data Types

Instead of adding properties to an object, an extension **MAY** reference a type defined entirely in an external specification (e.g. `External:RPP-JSContact-Profile:Card`), as described in [\[I-D.ietf-rpp-data-objects\]](#). This is the appropriate mechanism when substituting or supplying a whole, independently versioned data type, rather than adding fields to an existing RPP-defined object.

5.7. New RPP Result Codes

Extensions **MAY** define new RPP result codes within the reserved result code classes, registered in the RPP Result Codes registry defined in [\[I-D.ietf-rpp-core\]](#). New result codes **MUST** be registered in the RPP Result Codes registry defined in [\[I-D.ietf-rpp-core\]](#).

5.8. Error Object Extension Fields

Extensions **MAY** add new fields to the Problem Detail error object to convey additional, extension-specific information about the cause of an error, as described in [\[I-D.ietf-rpp-core\]](#).

TODO: there is no schema for the Problem Detail error object defined in the base specification, fix this?

5.9. Additional Query Parameters

Extensions **MAY** define additional HTTP query parameters for existing operations, for example to further qualify a check or read request, as described in [\[I-D.ietf-rpp-core\]](#). Any transient parameter for a read, query, update, and delete operation **MUST** be translated into an HTTP query parameter, named by the operation's parameter identifier.

5.10. New Authentication and Authorization Methods

TODO: is this an extension point?

5.11. Discovery document

New properties added by an extension to the RPP Discovery document **MUST** be included in the appropriate section of the document, following the structure defined in [\[I-D.ietf-rpp-core\]](#).

TODO: there is no schema for the Problem Detail error object defined in the base specification, fix this?

6. Extending Data Objects

RPP is designed so that extension specifications can add new data elements to data objects, component objects, and operations without modifying the specification that originally defines them, as described in [\[I-D.ietf-rpp-data-objects\]](#). This section defines the normative method an extension specification **MUST** follow when it adds properties to an object defined in a base specification, so that JSON Schema validation, as defined in [\[I-D.ietf-rpp-json\]](#), keeps functioning correctly across specifications.

6.1. JSON Schema Composition

Every data object defined in a base specification, when modified by an extension specification, must be extended using an extension specification. The extension specification **MUST** define a new JSON Schema document that composes the base object's schema with the new properties using `allOf`. The extension specification **MUST NOT** copy, redefine, or otherwise restate the JSON Schema definition of the base object. Instead, it **MUST** reference the base object's definition by its fully qualified `$id`-relative pointer (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) in an `allOf` branch, and add its own new properties in a separate branch.

An extension schema **MUST** declare which base object definitions it extends using the `rpp:extends` property, as defined in [\[I-D.ietf-rpp-json\]](#). The property maps the fully qualified reference of each base object definition to the local definition, within the extension schema, that contributes the new properties. This mapping allows an implementation to construct, for a given base object, the effective schema that combines the base definition with every supported extension, without any knowledge of the extensions in advance.

6.2. Schema Identification

A specification that defines JSON Schema for RPP objects **MUST** assign a stable `$id` to its schema document(s). Extension specifications **MUST** reference a base object's definition by its `$id` (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) rather than copying the base definition into their own schema.

6.3. Consumer Implementation

TODO: this section should probably be moved to the `rpp-json` document

A consumer, which can be a server or client, does not need to generate a distinct set of code classes for every base object and extension combination offered by a particular server. Because extension schemas are additive, self-contained, and independent of one another, a consumer implementation **SHOULD** instead maintain one set of code classes for

each base object and a separate set of code classes for each extension it supports, keeping an extension's classes distinct from the base object's classes rather than flattening their properties together. Composing a base object with whichever extensions apply then becomes a runtime decision, rather than something fixed by code generation for every individual server.

A consumer learns at runtime which extensions a given server supports from the extensions list in that server's RPP Discovery response, as defined in [I-D.ietf-rpp-core]. It SHOULD attach, populate, and validate extension class instances only for the extensions the server has advertised as supported, and SHOULD ignore any properties in a response that belong to an extension it does not recognize or does not support.

Because an extension that is not attached to a given base object instance contributes no properties at all, a consumer naturally serializes only the base object together with the properties of whichever extensions are actually in use, without needing to distinguish this case from a property that is merely absent or null within a supported extension. Attached extensions are merged flat with the base object on the wire, consistent with the schema composition described in "JSON Schema Composition". Within an attached extension, an optional property without a value continues to follow Rule 2 and is omitted from the output rather than represented as null.

6.3.1. Java

For Java clients, a generic library [RPP-JSON-JAVA-LIB] can implement the schema handling described in this document without any code that is specific to a particular extension. The library loads the RPP base schema and the extension schemas supported by a server, uses the `rpp:extends` mappings to compose the effective schema of a base object with `allOf`, applies the `unevaluatedProperties` injection described in "Validation" to the composed schema only, and validates JSON instances against the result. Adding support for a new extension then consists of adding its schema document to the library; no new classes are required.

6.4. Registration

An extension specification that adds properties to an existing object MUST register the added data elements in the Object and Operation Extension registry defined in [I-D.ietf-rpp-data-objects]. The registration MUST additionally include a dereferenceable URL to the JSON Schema document defining the `$defs` entry described in the Additive Schema Composition section above, and MUST identify the base object type(s) being extended by their registered identifier.

6.5. Validation

A deployment that supports a specific combination of extensions (a profile, as defined in [I-D.ietf-rpp-core]) MUST build, for every extended object type, a combined schema consisting of the base object's definition and the `allOf` branch contributed by every supported extension registered for that object type.

Before using a combined schema to validate a JSON instance, implementations MUST inject `"unevaluatedProperties": false` at every object-schema node of the combined schema, following the algorithm described in [I-D.ietf-rpp-json]. This injection MUST be performed only on the combined, per-deployment schema, and MUST NOT be present in the schema document published by any individual base or extension specification. This ensures undeclared properties are rejected while any supported combination of registered extensions validates successfully.

7. Extension Specification Format

An extension specification **SHOULD** follow the structure defined in this section, so that all extension specifications are uniform and can be read, reviewed and processed in the same way.

An extension specification **SHOULD** contain at least the following sections, in this order:

1. Introduction: the purpose of the extension, the base specifications it builds upon, and its motivation.
2. Terminology and Conventions.
3. Extension Overview: the name, the identifier, the version, the RPP version the extension is compatible with, the `$id` of the JSON Schema document, and the list of base objects that the extension changes.
4. Changed Data Objects: for every base data object, component object, process object or operation that the extension changes, the added data elements, associations and transient operation elements. Each added data element **MUST** be defined using the data element attributes defined in [\[I-D.ietf-rpp-data-objects\]](#) (name, identifier, cardinality, mutability, data type, description and constraints).
5. New Data Objects: every data object, component object or process object that the extension introduces, including its object description, data elements and operations.
6. JSON Schema: the single JSON Schema document of the extension.
7. Examples.
8. IANA Considerations.
9. Security Considerations.
10. Privacy Considerations
11. Internationalization Considerations.

The sections "Changed Data Objects" and "New Data Objects" **MUST** both be present. An extension that changes no existing object, or introduces no new object, **MUST** state "None" in the corresponding section. At least one of the two sections **MUST** define content.

The extension specification **MUST** define exactly one JSON Schema document, which contains the definitions for all changed and all new objects of the extension. The document **MUST** have a top-level `$id`, **MUST** declare the changed base objects using `rpp:extends` as described in "JSON Schema Composition", and **MUST NOT** contain `unevaluatedProperties` or `additionalProperties` set to `false`, see "Validation". An extension specification **MUST NOT** split its schema over multiple documents.

The extension specification **MUST** include examples. For every operation that the extension adds or changes, and for every changed or new object, the specification **MUST** include at least one valid JSON example of the request or response representation. Every valid example **MUST** validate successfully against the effective schema that is composed from the base schema and the JSON Schema document of the extension, as described in [Section 6.5](#).

8. IANA Considerations

TODO

9. Internationalization Considerations

TODO

10. Privacy Considerations

TODO

11. Change History

TODO

12. References

12.1. Normative References

- [I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>>.
- [I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>>.
- [I-D.ietf-rpp-json] Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>>.
- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <<https://www.rfc-editor.org/info/rfc3986>>.

12.2. Informative References

- [JSON-SCHEMA] JSON Schema, "JSON Schema: A vocabulary for describing JSON Data", 2020, <<https://json-schema.org/draft/2020-12/json-schema-core>>.
- [RPP-JSON-JAVA-LIB] SIDN Labs, "rpp-json-java-lib: Java library for RPP JSON Schema extensions", 2026, <<https://github.com/SIDN/rpp-json-java-lib>>.

Appendix A. Example Extension

This appendix is informative. It shows an extension specification that follows the structure defined in "Extension Specification Format", and it shows how the extension can be validated with a generic Java library. The example extension adds the property `extProperty1` to the Domain Name data object.

A.1. Extension Overview

Field	Value
Name	RPP Example Extension Property
Identifier	urn:ietf:params:rpp:extension:example-ext-property
Version	1.0
RPP version	1.0
Schema \$id	https://rpp.example/schemas/ext-domain-extproperty1.json
Changed base objects	domainName
New objects	None

Table 1

A.2. Changed Data Objects

A.2.1. Domain Name Data Object

- Identifier: domainName

The following data element is added to the Domain Name Data Object.

- Extension Property 1
 - Identifier: extProperty1
 - Cardinality: 0-1
 - Mutability: create-only
 - Data Type: String
 - Description: An example property that is added to the Domain Name Data Object by the extension.
 - Constraints:
 - The value MUST contain between 1 and 64 characters.
 - The value MUST be provided in the Create operation.

The Read operation returns extProperty1 when the domain name was created with a value for it. No other operations are changed.

A.3. New Data Objects

None.

A.4. JSON Schema

The extension defines a single JSON Schema document. The document declares, using rpp:extends, that it extends the create and read definitions of the base object, and contributes the new property in a separate definition:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/schemas/ext-domain-extproperty1.json",

  "rpp:extends": {
    "https://rpp.example/rpp/schema.json#/$defs/domainObject.create": "#/
    $defs/ext.domain.create",
    "https://rpp.example/rpp/schema.json#/$defs/domainObject.read": "#/$defs/
    ext.domain.read"
  },

  "$defs": {
    "ext.domain.create": {
      "type": "object",
      "properties": {
        "extProperty1": { "type": "string", "minLength": 1, "maxLength": 64 }
      },
      "required": ["extProperty1"]
    },
    "ext.domain.read": {
      "type": "object",
      "properties": {
        "extProperty1": { "type": "string" }
      }
    }
  }
}
```

A.5. Examples

The following instance is valid, because `extProperty1` is declared by the supported extension:

```
{
  "@type": "domainName",
  "name": "example.example",
  "registrant": { "@type": "contact", "id": "c-1234" },
  "nameservers": [
    { "@type": "host", "hostName": "ns1.example.net" }
  ],
  "extProperty1": "some value"
}
```

The following instances are invalid. The first omits the required `extProperty1`, the second gives `extProperty1` the wrong type, and the third contains `extProperty2`, which no supported extension declares and which is therefore rejected by the injected `unevaluatedProperties` keyword:

```
{ "@type": "domainName", "name": "example.example" }
```

```
{ "@type": "domainName", "name": "example.example", "extProperty1": 42 }
```

```
{
  "@type": "domainName",
  "name": "example.example",
  "extProperty1": "some value",
  "extProperty2": "not declared by any supported extension"
}
```

The same base schema and the same instance that uses `extProperty1` are rejected by a deployment that does not support the extension, since in that deployment `extProperty1` is an undeclared property. This is the behavior required by "Validation": the set of accepted properties is determined only by the extensions that the deployment supports.

A.6. Composition and Validation

For a deployment that supports the example extension, the effective schema for `domainObject.create` references the base definition and the extension definition, and has its own `$id`:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/rpp/effective/domain.create.json",
  "$ref": "#/$defs/effective.domainObject.create",
  "$defs": {
    "effective.domainObject.create": {
      "allOf": [
        { "$ref": "https://rpp.example/rpp/schema.json#/$defs/domainObject.create" },
        { "$ref": "https://rpp.example/schemas/ext-domain-extproperty1.json#/$defs/ext.domain.create" }
      ]
    }
  }
}
```

A.7. IANA Considerations

TODO:

A.8. Security Considerations

TODO:

A.9. Privacy Considerations

TODO:

A.10. Internationalization Considerations

TODO:

This example extension, its schema and the examples above can be validated with the generic Java library in the `rpp-json-java-lib` repository [[RPP-JSON-JAVA-LIB](#)]. The library loads the base schema and the extension schemas supported by a deployment, builds effective schemas from the `rpp:extends` mappings, applies the injection of `unevaluatedProperties` described in "Validation", and validates JSON instances against the result. The repository contains this example extension, the example instances and unit tests, and its README describes how to build and run them.

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink

SIDN Labs

Email: maarten.wullink@sidn.nl

URI: <https://sidn.nl/>

Pawel Kowalik

DENIC

Email: pawel.kowalik@denic.de

URI: <https://denic.de/>